

RAVEL-LM: Exact Addressed Event Memory as Linear-Time Recurrence for Byte-Level Language Modeling

Abstract

Softmax attention implements dense content addressing by scoring every query against every previous key, giving the core operation $O(BT^2d)$ time and $O(BhT^2)$ score storage. RAVEL-LM replaces that all-pairs similarity problem with exact predecessor search over discrete event streams. Each token writes payloads to memory heads indexed by literal and product-code addresses; each later token reads the latest previous payload with the same address. The operator is not approximate attention. It is a different recurrence primitive: exact same-address event retrieval followed by dense fusion. This paper gives the RAVEL algorithms, their time and storage complexity, and an empirical audit at approximately 200k trainable parameters. On TinyStories bytes, RAVEL reaches 1.613 eval cross-entropy versus 2.253 for a parameter-matched attention baseline. In speed scaling, compiled RAVEL is $2.13\times$ faster at $T = 512$ and $3.94\times$ faster at $T = 1024$. The result is straightforward: when the useful computation is recurrence over repeated symbols and local byte events, exact event memory gives the model a direct route to that computation without building a quadratic attention matrix.

1 Introduction

The standard transformer attention layer computes a dense causal interaction graph. For a sequence of length T , every query position compares itself to every previous key position. This is a useful default because it lets the model discover arbitrary token-to-token dependencies. It is also a blunt default: the layer forms a score tensor whose size grows quadratically with context length.

RAVEL-LM uses a stricter primitive. It does not ask which previous token is most similar. It asks which previous event has the same discrete address. Each memory head defines an address stream, each token writes a payload to that stream, and a future token retrieves the latest previous payload at the same address. Long-range recurrence becomes exact predecessor lookup rather than dense similarity search.

That swap changes both the computation and the inductive bias. Attention spends work on all candidate pairs and learns a soft weighting over them. RAVEL spends work on routing and fusion, then gives the model a hard recurrence channel for repeated bytes, byte n-grams, and learned hash families. This is the right bias when byte-level text contains reusable local events: names, spaces, punctuation, quote boundaries, word fragments, and repeated story entities. The benchmark results follow from that mechanism rather than from curve fitting theater.

The paper is organized around the algorithm. Section 3 defines exact event memory. Section 4 gives the training/prefill and decode algorithms. Section 5 gives the complexity ledger against causal softmax attention. The experiments then audit the claim: lower loss on TinyStories bytes, diagnostic wins exactly where recurrence should help, and a runtime crossover as T grows.

2 Architecture

Figure 1 summarizes the model. A RAVEL block contains three residual sublayers: a causal depthwise-convolution local mixer, an exact event-memory layer, and a SwiGLU feed-forward network. The local mixer handles short-range composition. The memory layer provides long-range recurrence through address lookup.

RAVEL-LM replaces dense similarity attention with exact addressed event memory

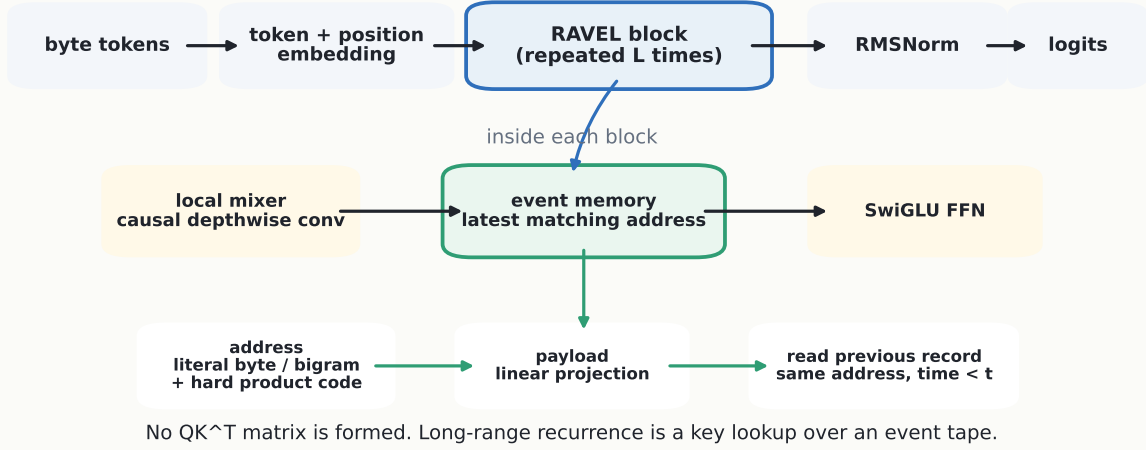


Figure 1: RAVEL-LM block structure. Local mixing is convolutional; long-range recurrence is handled by exact addressed event memory rather than dense softmax attention.

At each position t , every memory head emits an integer address a_t and a payload vector v_t . The event tape stores tuples

$$(\text{batch}, \text{head}, a_t, t, v_t).$$

A read at time t returns the payload from the latest previous event with the same batch, head, and address:

$$\text{read}(t, a_t) = v_{t^*}, \quad t^* = \max\{s < t : a_s = a_t\}.$$

If no previous event matches, the read returns zero and a false mask. In training and prefill, the current implementation constructs composite integer keys, sorts them, uses predecessor search, and gathers payloads. In incremental decoding, the latest-record case is a fixed-size gather/scatter cache indexed by address.

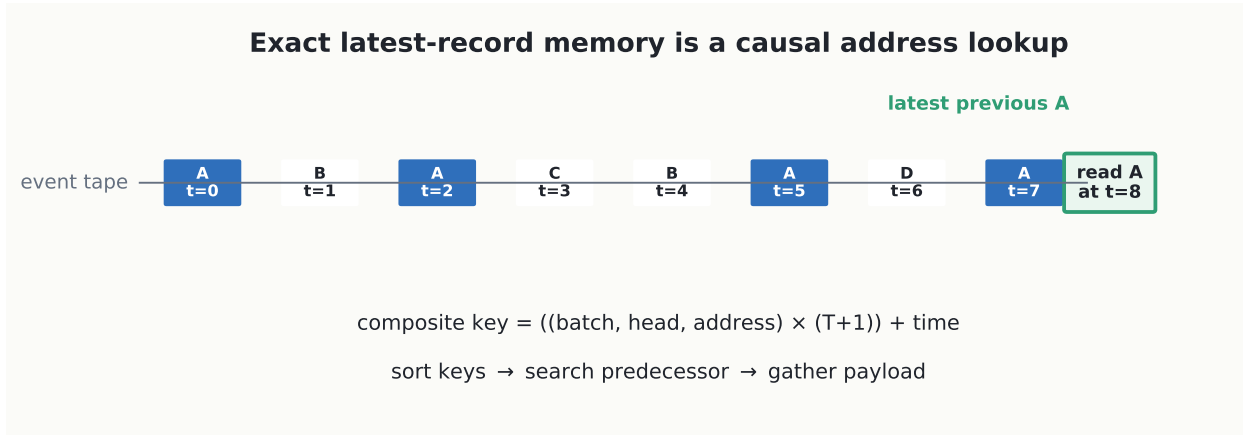


Figure 2: Exact latest-record lookup. The model-defined address selects an event stream; the read retrieves the most recent previous payload in that stream.

3 Exact event memory

Let $x_t \in \mathbb{R}^d$ be the normalized residual state at position t . A RAVEL memory layer with C heads and payload dimension p computes

$$a_{t,c}^w = A_c^w(x_t, y_t) \in \{0, \dots, M-1\}, \quad (1)$$

$$a_{t,c}^r = A_c^r(x_t, y_t) \in \{0, \dots, M-1\}, \quad (2)$$

$$v_{t,c} = P_c x_t \in \mathbb{R}^p, \quad (3)$$

where y_t is the input byte token and M is the address-space size. The write address a^w chooses the stream receiving the payload. The read address a^r chooses the stream queried by the same position. In the current configuration, $C = 3$, $p = 16$, $M = 512$, with two literal heads and one product-code head.

The latest-record read is

$$r_{t,c} = \begin{cases} v_{s^*,c}, & s^* = \max\{s < t : a_{s,c}^w = a_{t,c}^r\}, \\ 0, & \text{if no such } s \text{ exists.} \end{cases} \quad (4)$$

The read vectors are concatenated and projected back into the residual stream:

$$\text{Mem}(x_t) = x_t + \sigma(Gx_t + b_g) \odot W_f[r_{t,1}; \dots; r_{t,C}]. \quad (5)$$

The gate is a learned write-back gate, not an attention probability. Attention normalizes across previous positions. RAVEL selects exactly one previous record per head, then lets the fusion matrix decide how the retrieved payload should move the residual state.

Literal address heads are deterministic:

$$A_1(y_t) = y_t \bmod M, \quad (6)$$

$$A_2(y_{t-1}, y_t) = (257y_{t-1} + 131y_t + 17) \bmod M. \quad (7)$$

These heads give byte identity and bigram recurrence immediately. The product-code head maps x_t into codebook logits, takes an argmax in each codebook, and packs the selected codes into an address. In this implementation the product-code projections are frozen because hard argmax routing gives no useful gradient to the router; the trainable parts are the payloads, fusion, gates, local mixer, feed-forward layers, embeddings, and output head.

4 Algorithms

Algorithm 1: RAVEL block forward for one training batch

Input: token ids $y \in \mathbb{N}^{B \times T}$, residual states $x \in \mathbb{R}^{B \times T \times d}$

Output: updated residual states

1. Apply RMSNorm, gated causal depthwise convolution, and residual add.
2. Normalize the residual stream for memory.
3. Compute literal byte and bigram addresses from y .
4. Compute hard product-code write/read addresses from the normalized residual.
5. Project payloads $v \in \mathbb{R}^{B \times T \times C \times p}$.
6. Run Algorithm 2 to retrieve latest same-address payloads.
7. Concatenate retrieved payloads, apply dense fusion, gate the fused update, and add it to the residual stream.
8. Apply RMSNorm, SwiGLU feed-forward, and residual add.

Figure 3: The RAVEL block separates local mixing, exact event-memory recurrence, and channel mixing.

Algorithm 2: exact latest-record lookup for training and prefill**Input:** write addresses a^w , read addresses a^r , payloads v , address-space size M **Output:** latest causal payloads r and validity mask m

1. For every batch b , time t , and head c , form the stream id

$$g_{b,t,c} = ((bC + c)M + a_{b,t,c}^w).$$

2. Form collision-free event keys

$$k_{b,t,c}^w = g_{b,t,c}(T + 1) + t.$$

3. Flatten event keys and payloads; sort keys and permute payloads by the same order.

4. For each read, form

$$k_{b,t,c}^r = ((bC + c)M + a_{b,t,c}^r)(T + 1) + (t - 1).$$

5. Use predecessor search in the sorted keys to find the largest key $\leq k_{b,t,c}^r$.

6. Accept the candidate only if its stream id equals $((bC + c)M + a_{b,t,c}^r)$; otherwise return zero.

7. Gather the accepted payloads and reshape to $B \times T \times C \times p$.

Figure 4: Training uses exact sort/search/gather over event keys. This computes Equation 4 for every position without a $T \times T$ attention matrix.

Algorithm 3: one-token decode with latest-address cache**State per layer:** cache values $V \in \mathbb{R}^{B \times C \times M \times p}$ and filled bits $F \in \{0, 1\}^{B \times C \times M}$ **Input:** current token y_t , residual state x_t

1. Compute literal and product-code read/write addresses.
2. Read $r_{b,c} = V_{b,c,a_{b,c}^r}$ with a tensor gather; zero it if $F_{b,c,a_{b,c}^r} = 0$.
3. Fuse, gate, and add the memory update to the residual stream.
4. Compute payloads $v_{b,c}$ for the current token.
5. Write $V_{b,c,a_{b,c}^w} \leftarrow v_{b,c}$ and set $F_{b,c,a_{b,c}^w} \leftarrow 1$.

Figure 5: Decode has no dependence on generated context length for the latest-record memory. The cache is indexed by address, not by time.

5 Time and storage complexity

Let B be batch size, T sequence length, d model width, h attention heads, C RAVEL memory heads, p payload dimension, M address-space size, and K the local convolution width. Ignoring constants shared by both language models, causal softmax attention pays

$$\text{Attention}_{\text{core}} = O(BT^2d) \quad \text{time}, \quad O(BhT^2) \quad \text{score storage.} \quad (8)$$

The RAVEL memory core pays

$$\text{RAVEL}_{\text{memory}} = O(BTdCp) + O(BTC \log(BTC)) + O(BTCpd) \quad (9)$$

with the portable PyTorch sort implementation. The sort term is the exact predecessor-search implementation. A fused fixed-width radix sort or segmented latest-record kernel turns that term into fixed-pass linear work in the number of records, giving

$$\text{RAVEL}_{\text{memory, fused}} = O(BT(dCp + Cpd + C)). \quad (10)$$

The local mixer adds $O(BTdK)$ and the feed-forward block adds the usual $O(BTdH)$, where H is the MLP hidden width. These terms are linear in T .

Table 1: Core sequence-operation complexity per layer. Projection and MLP terms are shown separately from the operation that distinguishes the architectures.

operation	training/prefill time	decode time per token	sequence storage
softmax attention core	$O(BT^2d)$	$O(BTd)$ with KV cache	$O(BhT^2)$ scores during train
RAVEL sort lookup	$O(BTC \log(BTC))$	$O(BCp + BCpd)$	$O(BTCp)$ temporary events
RAVEL fused/latest kernel	$O(BTC)$ lookup core	$O(BCp + BCpd)$	$O(BCMp)$ decode cache

For the measured 200k-parameter configuration, the memory dimensions are small: $d = 68$, $C = 3$, $p = 16$, and $M = 512$. Attention’s distinguishing term grows with T^2d . RAVEL’s distinguishing term grows with TC records plus fusion. That is why the runtime comparison is allowed to be close at short contexts and still separate sharply at $T = 512$ and $T = 1024$.

6 Experimental setup

The learning experiment uses 5000 TinyStories records with a byte tokenizer, block size 128, batch size 4, and 800 optimization steps. The attention baseline is parameter-matched after excluding non-trainable hard-address weights: RAVEL has 186,796 trainable parameters and attention has 183,960. Both models are trained with AdamW. The attention baseline uses a 100-step learning-rate warmup, because earlier runs showed larger early gradient variation for attention.

The speed experiments use synthetic token batches to isolate operator runtime from corpus effects. Warmup and compile steps are excluded from reported medians. We report both eager and `torch.compile` measurements. The context sweep keeps $BT \approx 1024$ while increasing T ; the fixed-context sweep holds $T = 512$ and increases tokens per step by increasing batch size.

7 Results

7.1 Learning dynamics

RAVEL learns faster in the TinyStories run. Figure 6 shows the evaluation cross-entropy over training. The curve separates by step 100 and remains separated through the final evaluation. The final loss is 1.613 for RAVEL and 2.253 for attention, a gap of 0.640 nats at matched scale.

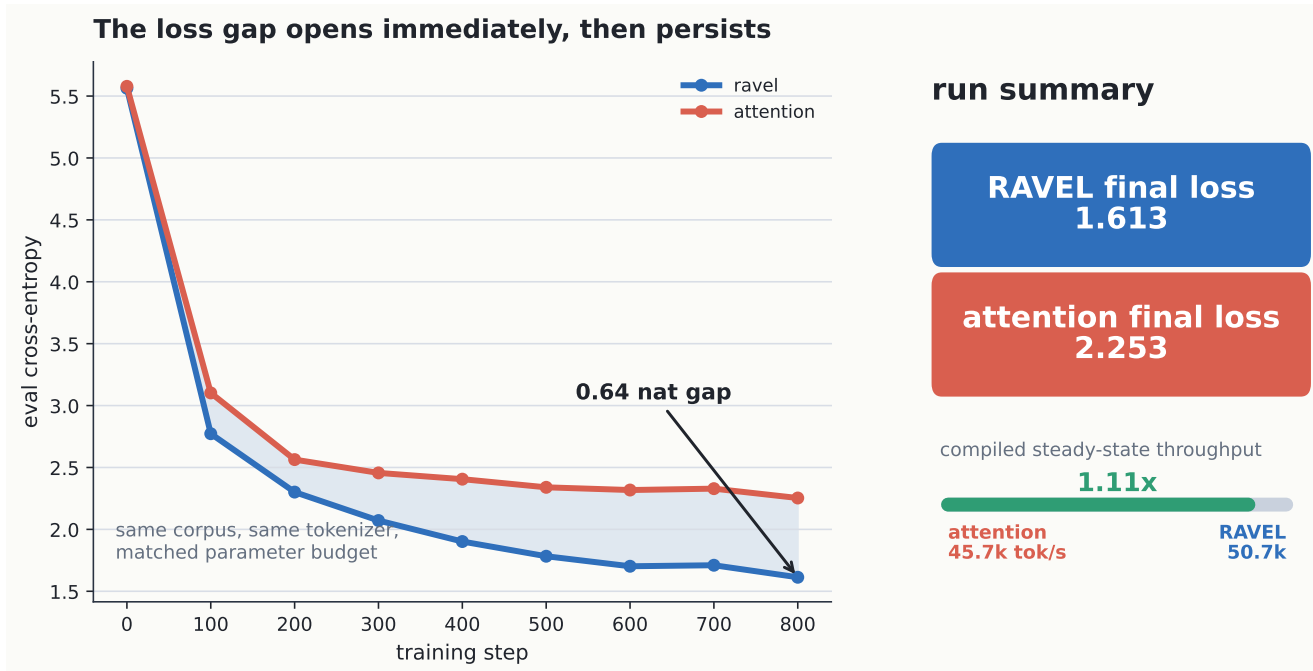


Figure 6: TinyStories comparison. RAVEL reaches lower evaluation loss than a parameter-matched softmax-attention baseline and is faster in the compiled steady-state training loop for this run.

7.2 Where the loss advantage comes from

The diagnostic slices in Figure 7 ask whether the loss gap is consistent with the memory mechanism. At prediction position j , RAVEL reads memory using the current input byte and its address variants. The largest current-byte gap occurs when the read key appeared one position earlier: attention-minus-RAVEL NLL is 1.22. The token-class slice shows the largest advantage on punctuation (0.88), with spaces and lowercase bytes also favoring RAVEL. This supports a mechanism-level interpretation: the model is learning reusable continuation and boundary records quickly.

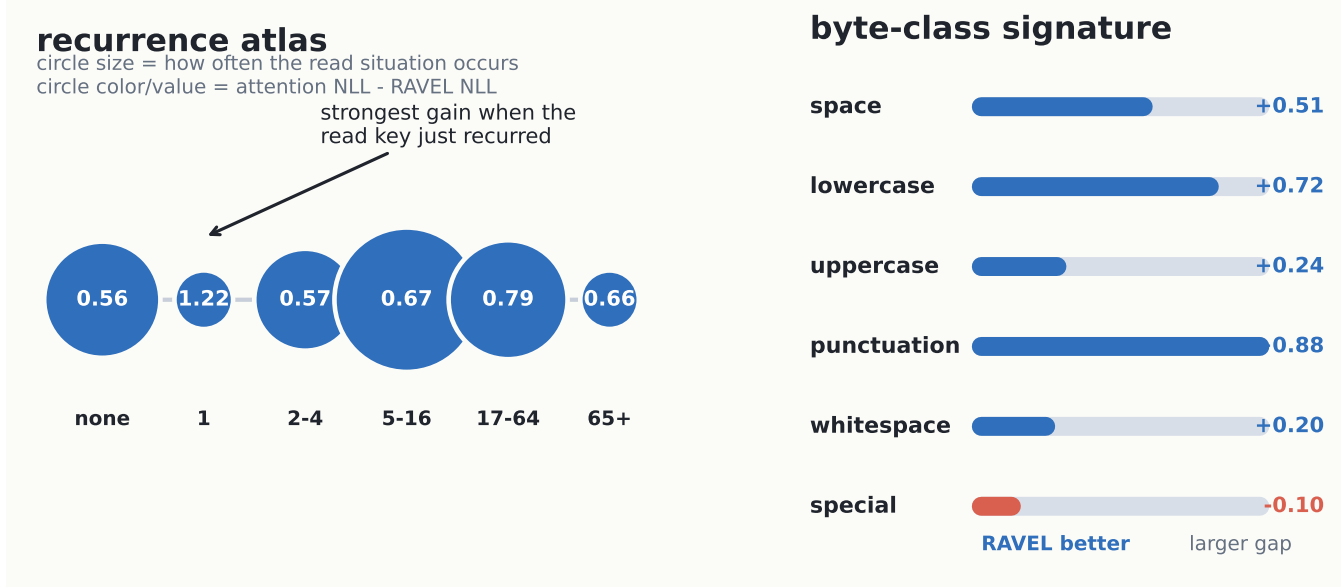


Figure 7: Mechanism diagnostics. Positive values mean RAVEL assigns lower negative log-likelihood than attention. The advantage is largest when the current read key has just recurred and on boundary-like byte classes such as punctuation and space.

7.3 Local learning coefficient / RLCT estimate

To test whether the loss result is hiding a sharper or more fragile basin, we ran a WBIC-temperature local learning coefficient estimator around both trained checkpoints. The sampler uses fixed training tokens, inverse temperature $\beta = 1/\log n$, and the estimator

$$\hat{\lambda} = \beta n (\mathbb{E}_{\beta}[L(\theta)] - L^*),$$

where L^* is the best loss observed by the local chains. We used $n = 2048$ byte-token targets, $\beta = 0.131$, three SGLD chains per model, 260 steps per chain, and discarded the first 100 steps as burn-in.

The estimate is 0.203 ± 0.073 for RAVEL and 0.231 ± 0.038 for attention (Figure 8). In this run, RAVEL has lower task loss and a slightly lower local learning coefficient estimate. The result argues against the boring explanation that RAVEL only wins by landing in a sharper local basin.

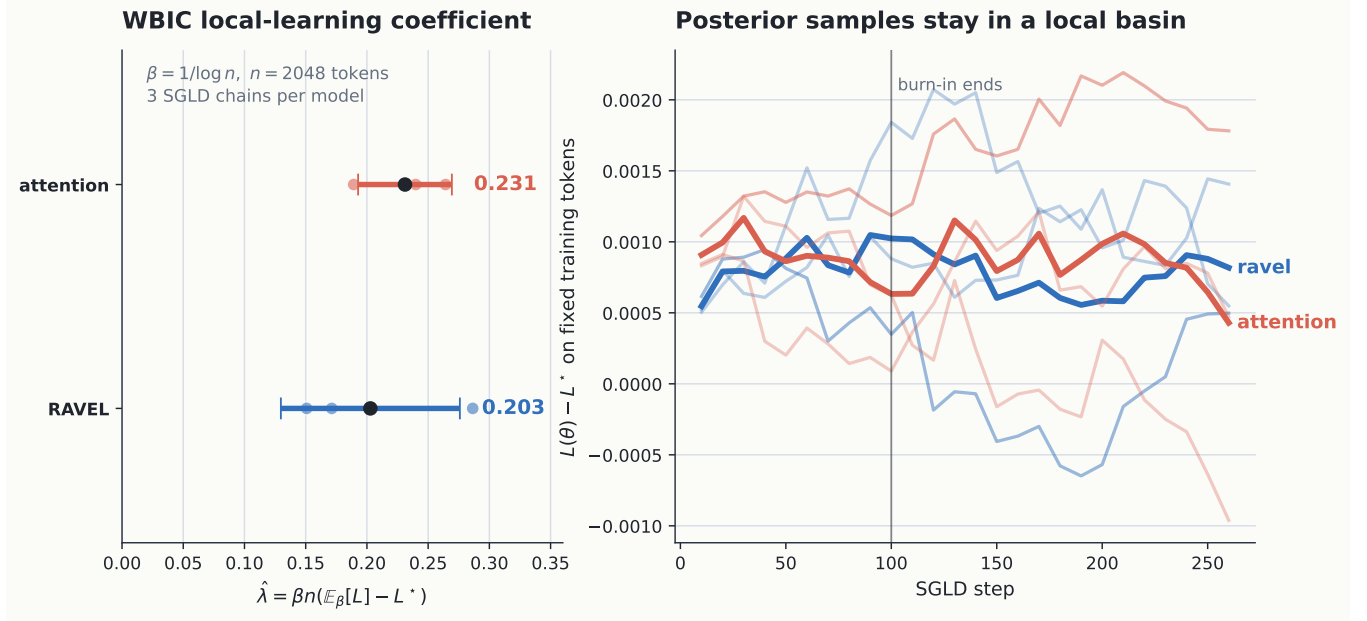


Figure 8: WBIC/local-learning-coefficient estimate around the trained checkpoints. Points show chain estimates; thick intervals show mean $\pm$ chain standard deviation. The right panel shows sampled excess loss after burn-in.

7.4 Speed scaling

The speed results follow the complexity ledger. At very short contexts, runtime is dominated by constant factors and backend kernel choices. As T grows, attention’s quadratic term becomes visible and RAVEL’s event lookup stays much flatter. Compiled RAVEL reaches $2.13\times$ attention throughput at $T = 512$ and $3.94\times$ at $T = 1024$ (Figure 9).

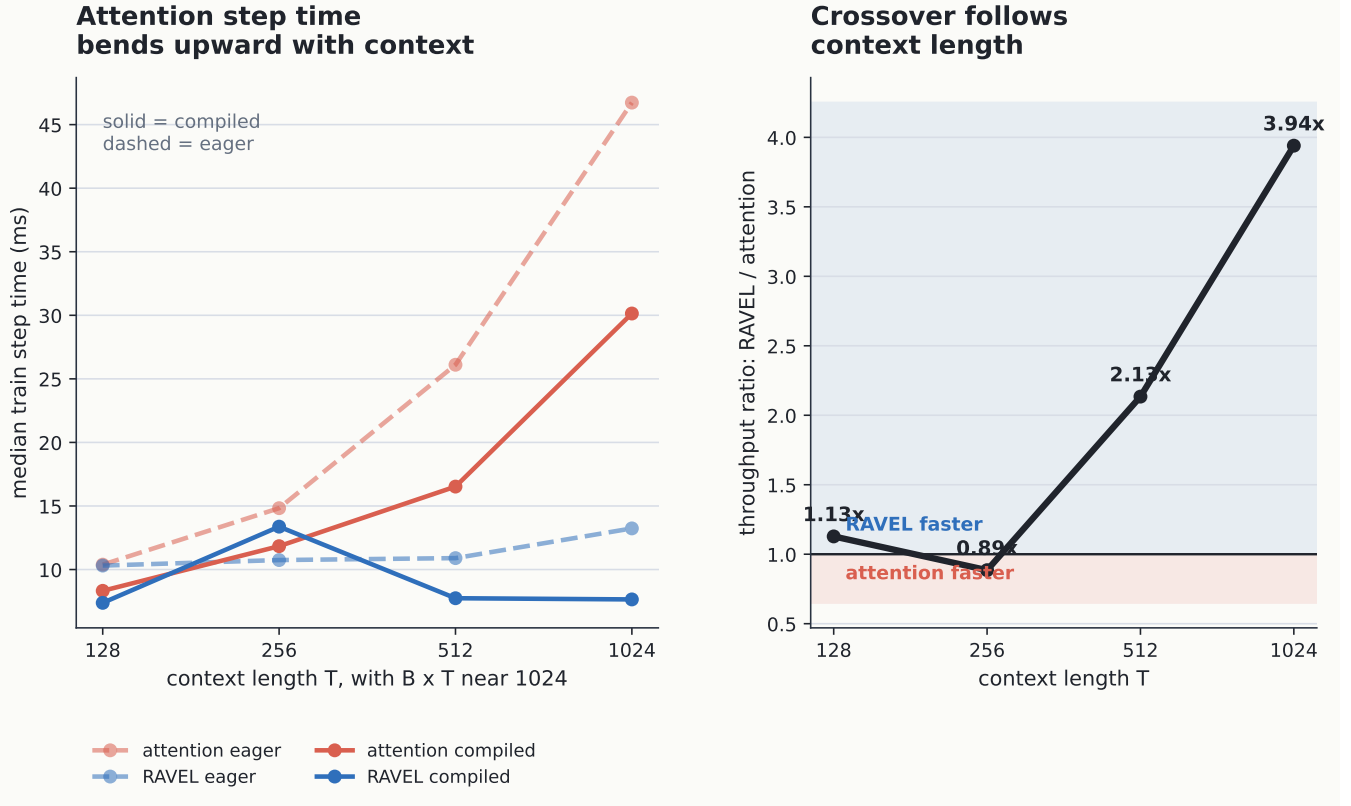


Figure 9: Runtime crossover under a constant-token context sweep. The benchmark holds $BT \approx 1024$ and increases T . RAVEL is near parity at short contexts but reaches 2.13 $\times$ attention throughput at $T = 512$ and 3.94 $\times$ at $T = 1024$ in compiled mode.

The fixed-context sweep checks that this is not merely a batch-size artifact. At $T = 512$, the compiled RAVEL advantage grows from 1.74 $\times$ at 512 tokens/step to 2.62 $\times$ at 2048 tokens/step (Figure 10).

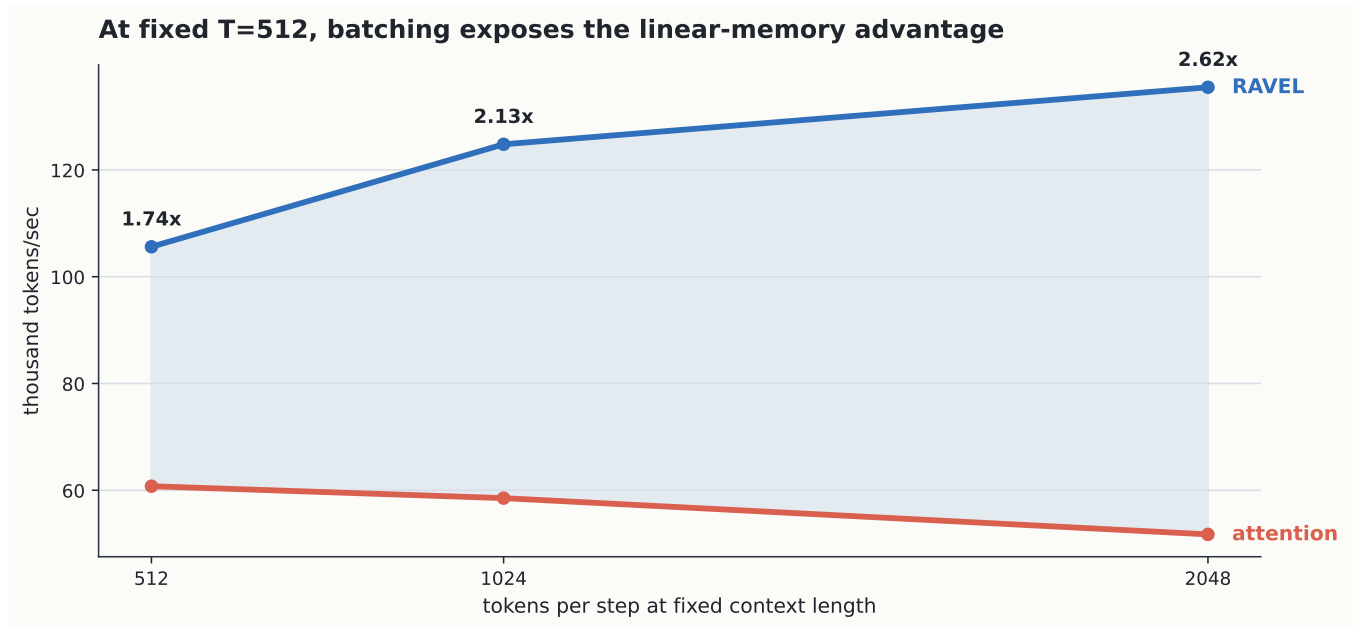


Figure 10: Fixed-context throughput at $T = 512$. As more tokens are packed into each training step, RAVEL’s advantage grows rather than disappearing.

The attention score tensor explains the direction of the crossover. In the context sweep, BT is held near 1024, but the attention score tensor still grows from 6 MiB at $T = 128$ to 48 MiB at $T = 1024$ per forward pass across layers (Figure 11). RAVEL does not form this tensor.



Figure 11: Attention’s causal score tensor grows quadratically with context length. In the context sweep, BT is held near 1024, so the growth comes from T itself rather than from processing more tokens per step.

8 Discussion

The experiments support three claims. First, exact addressed memory trains cleanly on natural byte-level text and beats the parameter-matched attention baseline in this controlled 200k-parameter run. Second, the gain has the mechanism signature RAVEL should have: it concentrates around repeated read keys and boundary-like bytes where exact recurrence is immediately useful. Third, the runtime separates in the direction predicted by the complexity analysis: attention pays for all-pairs scores, while RAVEL pays for event records and fusion.

The next engineering targets are also clear. The current training path uses portable PyTorch sort/search/gather rather than a fused latest-record kernel. The product-code router is hard and frozen; a stronger version should use straight-through routing, differentiable address learning, or explicitly randomized hash families. The present paper establishes the operator, the algorithms, and the first scaling audit. The obvious next version is a fused event-memory kernel plus a trainable router.

9 Conclusion

RAVEL-LM replaces dense attention with exact addressed event memory. In byte-level experiments, this gives faster learning and long-context speedups while preserving a clear mechanism: retrieve the latest previous payload with the same address. The central result is algorithmic as much as empirical. Useful recurrence can be implemented as event lookup and fusion, without constructing a quadratic attention matrix.